Guía de despliegue de FastAPI - Parte 1: Servidores ASGI

FastAPI es una aplicación ASGI, por lo que en producción necesita un servidor ASGI. Uvicorn es el servidor de referencia: es rápido, está basado en uvloop y httptools, y se ejecuta con el comando uvicorn main:app --host 0.0.0.0 --port 8000. La opción --reload solo debe usarse en desarrollo, porque vigila los archivos del proyecto y reinicia el proceso ante cada cambio, lo que consume recursos y puede provocar reinicios inesperados.

Un único proceso de Uvicorn utiliza un solo núcleo de CPU. Para aprovechar varios núcleos se ejecutan múltiples workers: uvicorn main:app --workers 4. Cada worker es un proceso independiente con su propia memoria, por lo que un modelo cargado en el lifespan se replica en cada uno. Una regla práctica habitual es usar un worker por núcleo disponible y ajustar según la memoria. Otra opción clásica es Gunicorn como gestor de procesos con la clase de worker uvicorn.workers.UvicornWorker, que aporta reinicio automático de workers caídos y control de timeouts.

El comando fastapi run, incluido en fastapi[standard], ofrece una forma simplificada de iniciar la aplicación en modo producción con Uvicorn, mientras que fastapi dev la inicia en modo desarrollo con recarga automática. Ambos detectan automáticamente el objeto app dentro de main.py o app/main.py.

Parte 2: Contenedores Docker

La forma más común de desplegar FastAPI es empaquetarla en una imagen Docker. Un Dockerfile típico parte de una imagen oficial liviana como python:3.12-slim, copia primero el archivo requirements.txt, ejecuta pip install --no-cache-dir -r requirements.txt y solo después copia el código de la aplicación. Este orden aprovecha la caché de capas de Docker: si solo cambia el código, no se vuelven a instalar las dependencias.

La instrucción final debe usar la forma exec (lista JSON): CMD ["fastapi", "run", "app/main.py", "--port", "80"]. La forma exec garantiza que el proceso reciba correctamente las señales de apagado (SIGTERM) y que el lifespan pueda ejecutar su código de limpieza. Si la aplicación corre detrás de un proxy que termina TLS, como Nginx o Traefik, se agrega la opción --proxy-headers para que Uvicorn confíe en los headers X-Forwarded-For y X-Forwarded-Proto.

En orquestadores como Kubernetes es preferible ejecutar un solo proceso Uvicorn por contenedor y dejar la replicación al orquestador (réplicas del Deployment), en lugar de usar --workers dentro del contenedor. Así el orquestador controla la escala, el balanceo y el reinicio de forma uniforme. Se recomienda exponer un endpoint de salud, por ejemplo /health, para las liveness y readiness probes.

Parte 3: Configuración y producción

La configuración debe leerse desde variables de entorno, nunca escribirse en el código. El paquete pydantic-settings ofrece la clase BaseSettings: se declara una clase Settings con campos tipados (database_url, secret_key, debug) y Pydantic los lee y valida desde el entorno o desde un archivo .env. Envolver la creación en una función decorada con functools.lru_cache evita releer el archivo en cada petición, y exponerla como dependencia permite sobrescribirla en los tests.

Detrás de un proxy que publica la aplicación bajo un subpath, por ejemplo /api/v1, se usa el parámetro root_path (o la opción --root-path de Uvicorn) para que la documentación y las URLs generadas apunten a la ruta correcta. Para HTTPS se recomienda que el proxy o el balanceador de carga gestione los certificados, por ejemplo con Let's Encrypt, y que la aplicación reciba tráfico HTTP interno.

Checklist de producción: desactivar --reload; definir workers o réplicas según la CPU; configurar logs estructurados; exponer /health; limitar CORS; fijar versiones de dependencias en requirements.txt; ejecutar el contenedor con un usuario no root; y monitorear latencia y errores con herramientas como Prometheus y OpenTelemetry.